KALI LINUX LAB REPORT

System Investigation, Documentation, Networking and Troubleshooting

Chapter 6 & 7

Platform: Kali GNU/Linux Rolling 2026.2 in VirtualBox
User: kali
Date: 17 October 2026

> **Guiding principle:** observe, investigate, record and verify, instead of changing system settings without understanding the cause.

At a Glance

Area	Result
Operating system	Kali GNU/Linux Rolling 2026.2 (`kali-rolling`, Debian-based)
Kernel	6.19.14+kali-amd64, x86_64
CPU / Memory	1 vCPU (Intel Core i7-1165G7, KVM hypervisor) / 1.9 GiB RAM, 1.1 GiB swap
Root disk	`/dev/sda1`, 19 GB, 86% used (2.6 GB free)
Network	`eth0` UP, `10.0.2.15/24`, gateway `10.0.2.2`
Connectivity	Loopback, `1.1.1.1`, `www.google.com` ping and HTTP 200 all succeeded
Main troubleshooting case	Evidence file failed to save because of a wrong working directory; fixed and verified
Log findings	Graphics driver, keyring and Evolution service messages documented, not "fixed"

1. Lab Workspace

The workspace was created with one command and then verified with `pwd` and `ls -la`:

```
mkdir -p ~/ch67lab/{evidence,reports,notes,files}
cd ~/ch67lab/
pwd        # /home/kali/ch67lab
ls -la
```

Directory	Purpose
`evidence/`	Raw command output and logs
`reports/`	Written reports and baseline files
`notes/`	Research notes
`files/`	Scripts and supporting files

Separating raw evidence from written explanations keeps the investigation organised and easy to audit.

Identity check

`whoami` returned `kali`, and `id` showed `uid=1000(kali)` with membership in the `sudo`, `adm`, `wireshark`, `netdev`, `kaboxer` and `vboxsf` groups. `sudo -v` was used to confirm administrative access. The output of `whoami` and `id` was also saved to `reports/baseline.txt` using `>` and `>>`.

2. Inspecting the Linux System

Command	Observation
`cat /etc/os-release`	Kali GNU/Linux Rolling, version 2026.2, `ID_LIKE=debian`
`uname -r`	`6.19.14+kali-amd64`
`uname -a`	`Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64`
`hostname`	`kali`
`uptime`	Up 6 minutes, 1 user, load average 0.07, 0.32, 0.21
`lscpu`	x86_64, 1 CPU, Intel Core i7-1165G7 @ 2.80 GHz, hypervisor vendor KVM, full virtualization
`free -h`	1.9 GiB total, 780 MiB used, 1.2 GiB available; swap 1.1 GiB, 0 B used
`df -h`	`/dev/sda1` 19 GB, 15 GB used, **86%**
`lsblk`	`sda` 20 GB with `sda1` (18.9 GB, `/`) and `sda5` (1.1 GB, swap); `sr0` optical drive

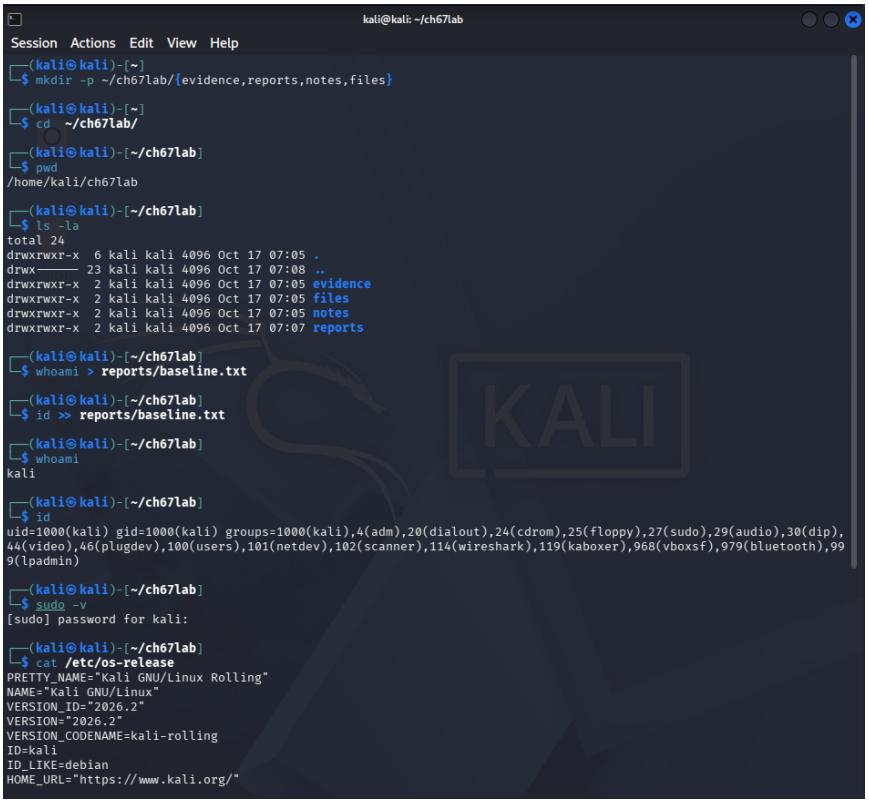


Figure 1: Workspace creation, identity check and OS release.

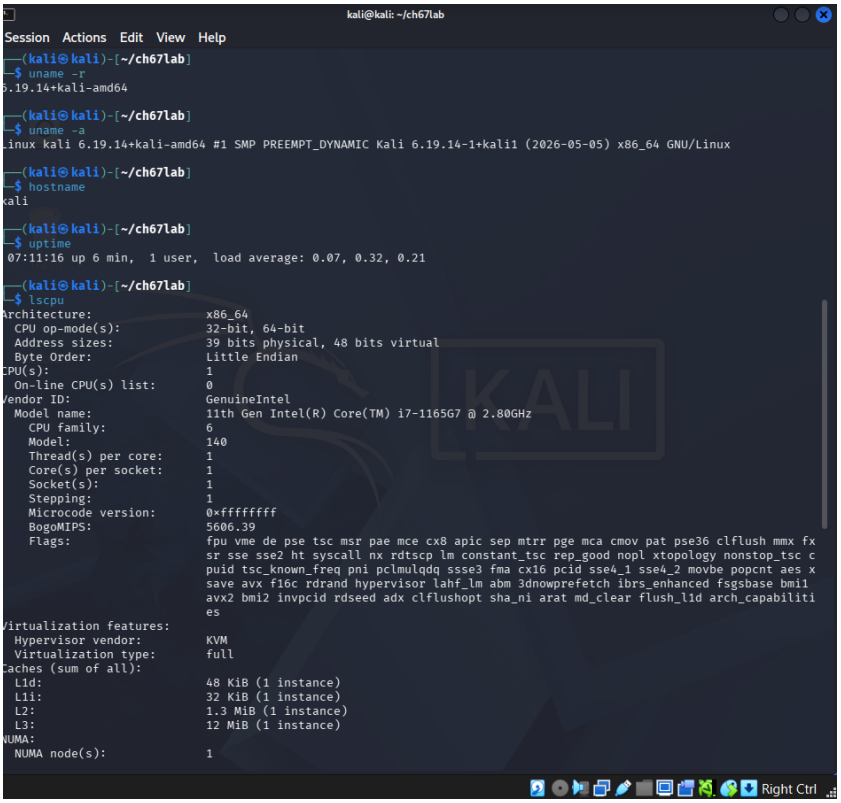


Figure 2: Kernel, hostname, uptime and CPU information.

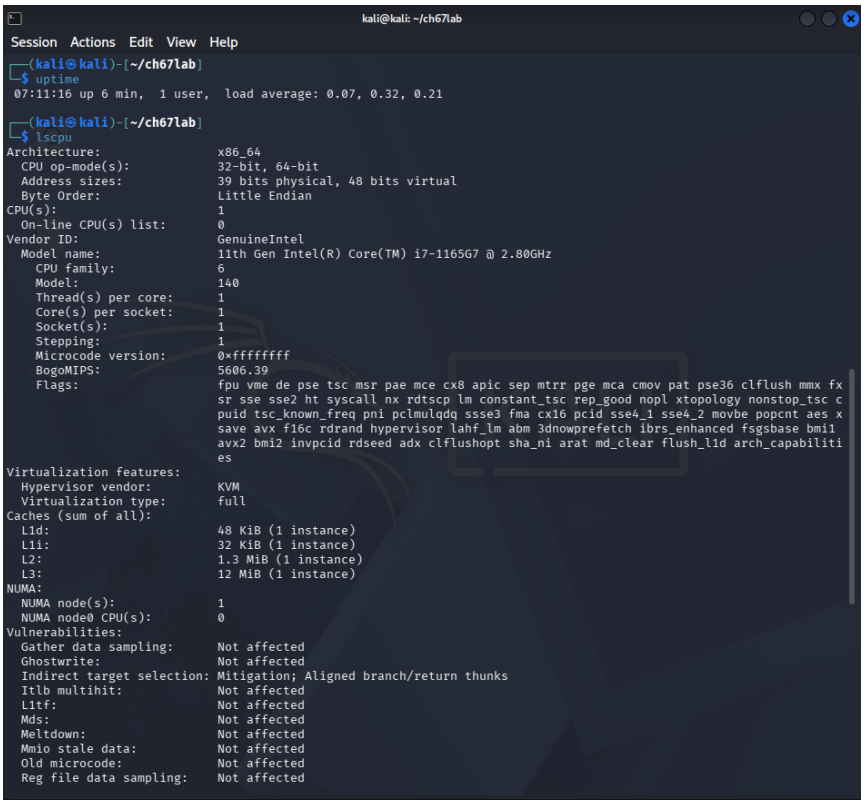


Figure 3: CPU vulnerability status from lscpu.

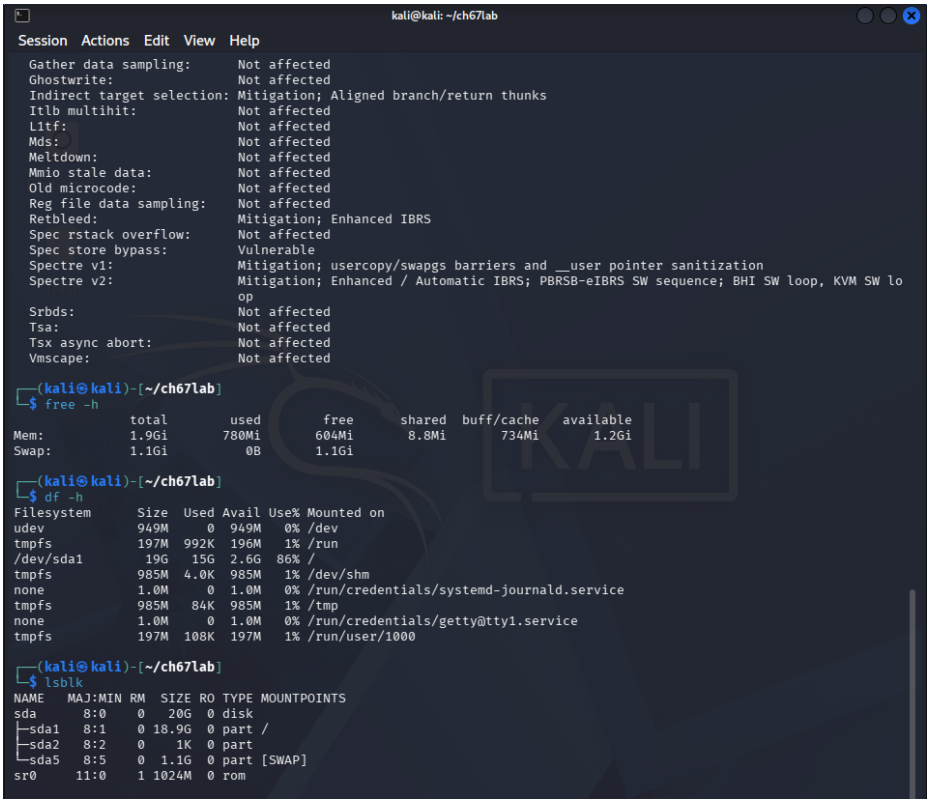


Figure 4: Memory, filesystem usage and block devices.

Notes

- The kernel version is separate from the Kali release number: Kali is the distribution, the kernel manages hardware.
- `lscpu` reported most CPU vulnerabilities as *Not affected* or *Mitigated* (Meltdown not affected, Spectre v1 and v2 mitigated, Retbleed mitigated with Enhanced IBRS). **Spec store bypass** was listed as *Vulnerable*.
- At 86% usage, the root filesystem is the most likely resource to cause trouble later, for example during package installs.

Baseline snapshot

A grouped command block captured the date, hostname, OS release, kernel, uptime, memory, disk and network details into one file:

```
{ date; hostname; cat /etc/os-release; uname -r; uptime; free -h; df -h; ip -br addr; ip route; } >
    reports/baseline.txt 2>&1
cat reports/baseline.txt
```

The snapshot was timestamped **Sat Oct 17 07:16:19 AM EDT 2026** and recorded the system after 11 minutes of uptime.

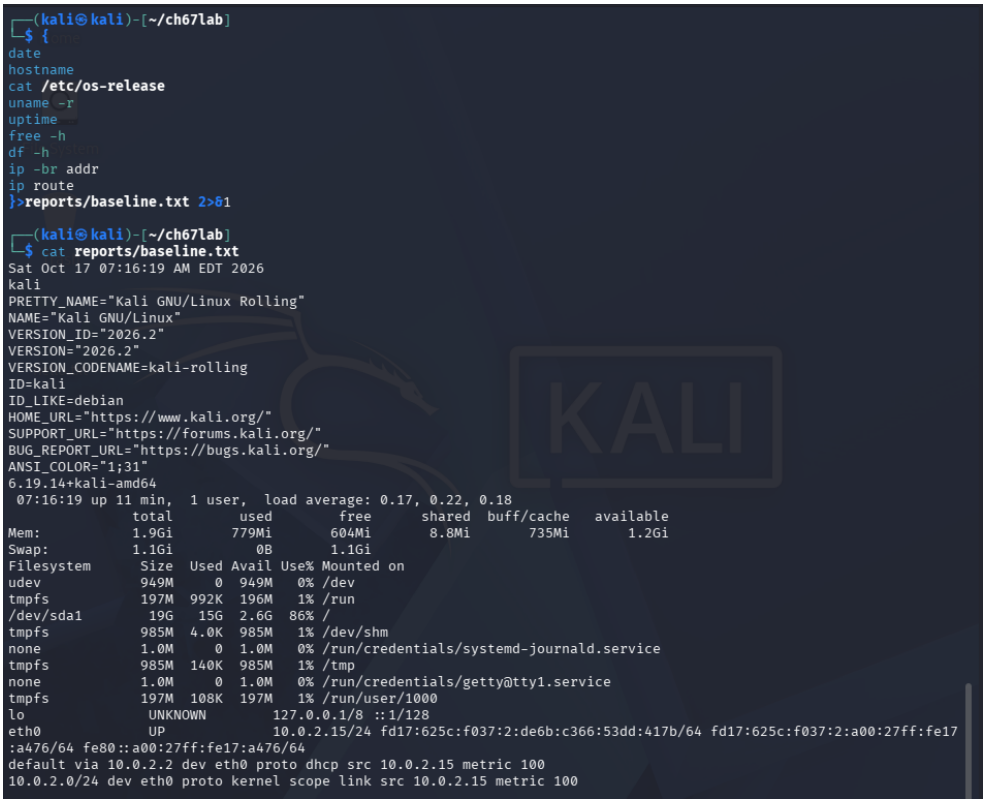


Figure 5: Baseline report saved to reports/baseline.txt.

3. Documentation and Package Information

Linux provides several layers of built-in documentation: `man`, `--help`, `apropos`, and package metadata through `apt show` and `dpkg -s`.

`dpkg -s openssh-client` confirmed the local package state:

Field	Value
Status	`install ok installed`
Version	`1:10.3p1-4`
Architecture	amd64
Maintainer	Debian OpenSSH Maintainers
Provides	`ssh-client` (ssh, scp, sftp, ssh-agent, ssh-add, ssh-keygen, ssh-keyscan)
Homepage	https://www.openssh.com/

Listing `/usr/share/doc/openssh-client/` showed `changelog.Debian.gz`, `changelog.gz`, `copyright`, `examples`, `NEWS.Debian.gz`, `OVERVIEW.gz`, `README`, `README.Debian.gz`, `README.dns` and `README.tun.gz`.

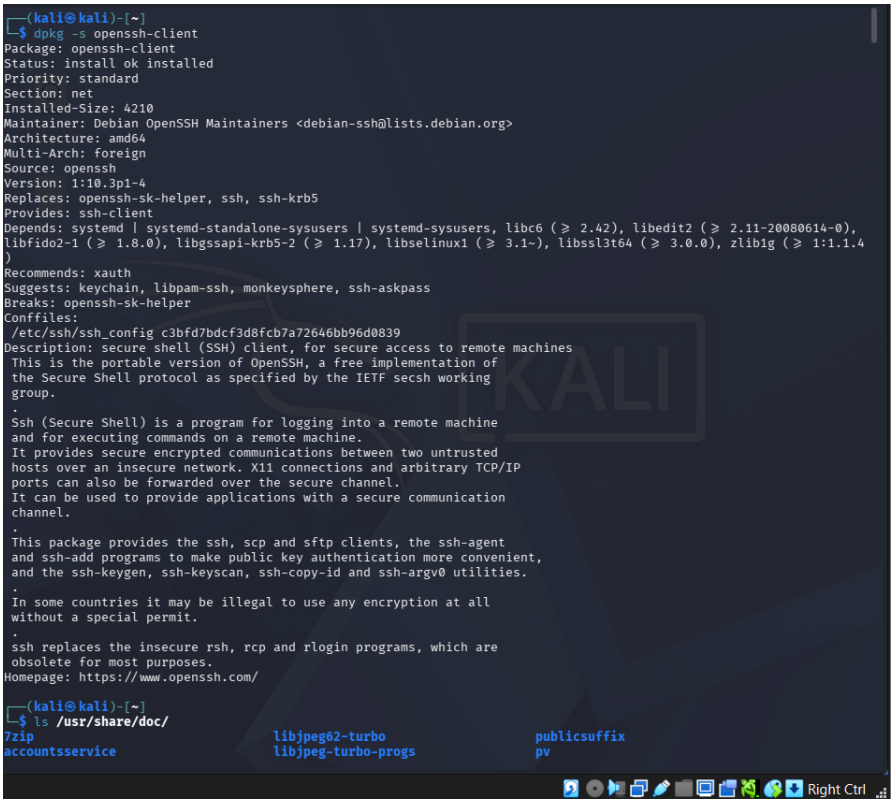


Figure 6: Package status of openssh-client.

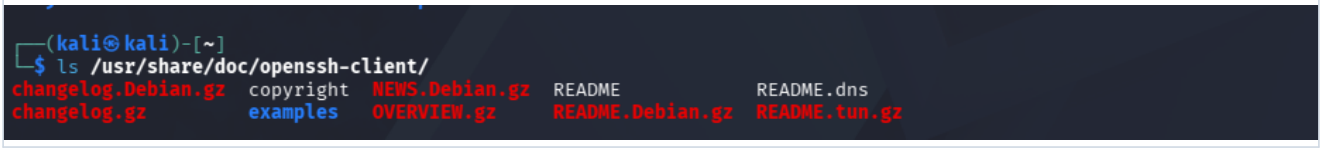


Figure 7: Installed OpenSSH documentation files.

> `apt show` reads package metadata from configured repositories, while `dpkg -s` reports what is actually installed on this machine.

4. Network Investigation

The network was tested layer by layer, from interface to application.

Layer	Command	Result
Interface and IP	`ip -br addr`	`lo` UNKNOWN `127.0.0.1/8`; `eth0` **UP** `10.0.2.15/24` plus IPv6 addresses
Routing	`ip route`	`default via 10.0.2.2 dev eth0 proto dhcp`; local subnet `10.0.2.0/24`
Addresses	`hostname -I`	`10.0.2.15` and two IPv6 addresses
DNS	`cat /etc/resolv.conf`	Generated by NetworkManager; nameservers `8.8.8.8`, `8.8.4.4`, `fd17:625c:f037:2::3`; search domain `home.arpa`
Local stack	`ping -c 4 127.0.0.1`	4/4 received, 0% loss, avg 0.666 ms
External IP	`ping -c 4 1.1.1.1`	4/4 received, 0% loss, avg 10.622 ms
Name + ICMP	`ping -c 3 www.google.com`	Resolved to `142.251.152.119`; 3/3 received, 0% loss, avg 70.5 ms
HTTP	`curl -I --max-time 10 www.google.com`	**HTTP/1.1 200 OK** from `gws`

Layer	Command	Result
Management	`nmcli device status`	`eth0` ethernet **connected** (Wired connection 1); `lo` loopback connected (externally)

The `10.0.2.0/24` network with gateway `10.0.2.2` is the typical VirtualBox NAT arrangement, so the VM reaches the internet through the host.

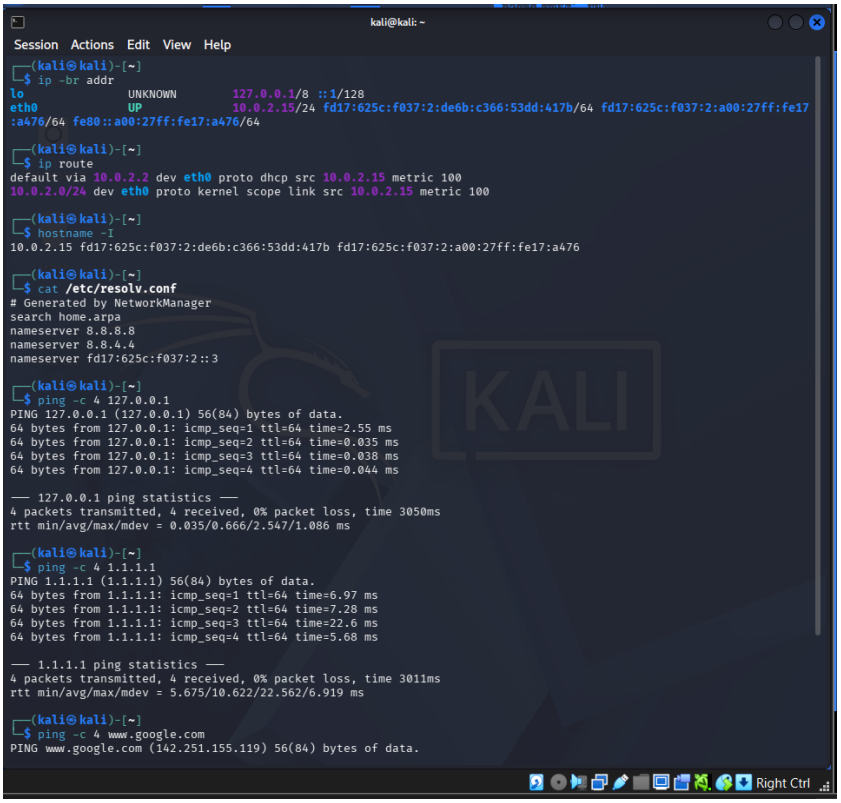


Figure 8: Interfaces, routing, DNS and ping tests.



Figure 9: HTTP test, service checks and nmcli device status.

Typing mistakes and what they taught

- `curl -I --max-time 10 hhttps://www.google.com` returned `Protocol "hhttps" not supported`. The error pointed straight at the typo, and the corrected request returned HTTP 200.
- `systemctl is-active NetworkManger` (missing the letter *a*) returned `inactive`. Because `resolv.conf` is generated by NetworkManager and `nmcli` reported the device as connected, the result reflected the misspelled service name and not a real outage. Cross-checking tests against each other prevented a wrong conclusion.
- `systemd-networkd` reported `inactive`, which is consistent with NetworkManager being the active network manager.

5. System Log Investigation

The traditional `/var/log/syslog` file did not appear in the `/var/log` listing. The directory held service logs (apache2, nginx, mysql, postgresql, samba, openvpn, lightdm, Xorg and others) alongside a `journal/` directory. This is normal on a systemd system, where logs are read with `journalctl`.

```
ls -lah /var/log/
journalctl -b -n 50 --no-pager
journalctl -b -p warning --no-pager
journalctl -b -p err --no-pager
```

Recent activity (`-n 50`) showed time synchronisation through `systemd-timesyncd`, routine root cron sessions and `phpsessionclean.service` starting.

Warning-priority entries included:

- APIC calibration not consistent with the PM-Timer (103 ms instead of 100 ms)
- `ehci_hcd` should always be loaded before `uhci_hcd` and `ohci_hcd`
- `colord`: failed to get EDID data, EDID length is too small
- `wireplumber`: UPower and BlueZ services unavailable
- `haveged.service`: referenced but unset variable `DAEMON_ARGS`
- Several `[UFW BLOCK]` entries for multicast UDP traffic to port 3702 on `eth0`, showing the firewall dropping discovery traffic
- `clocksource: Long readout interval, skipping watchdog check`

Error-priority entries are analysed below.

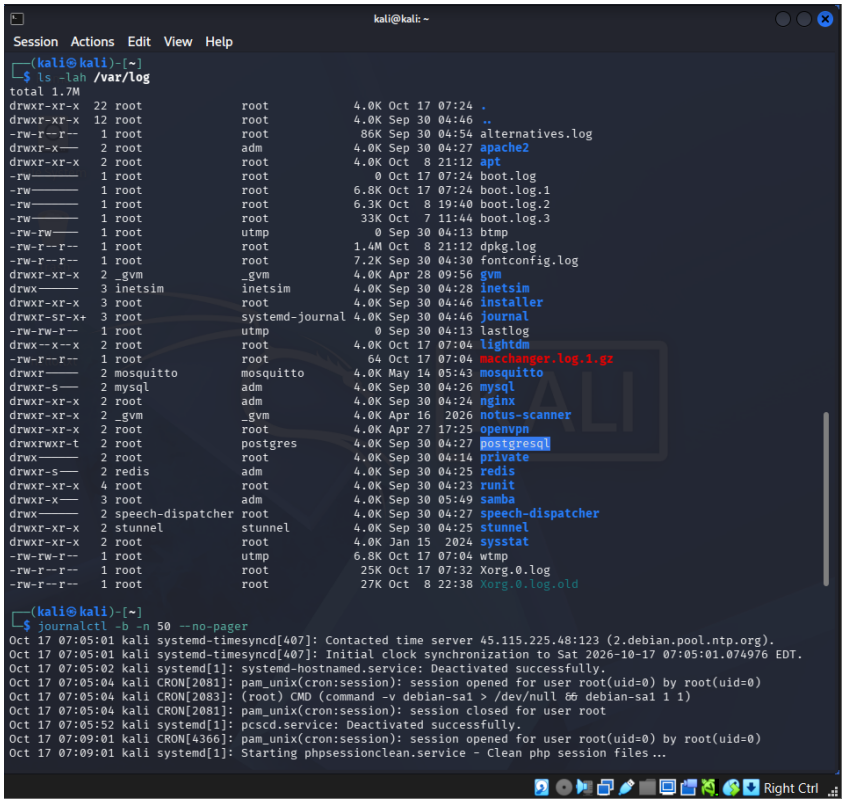


Figure 10: Log directory and recent journal entries.

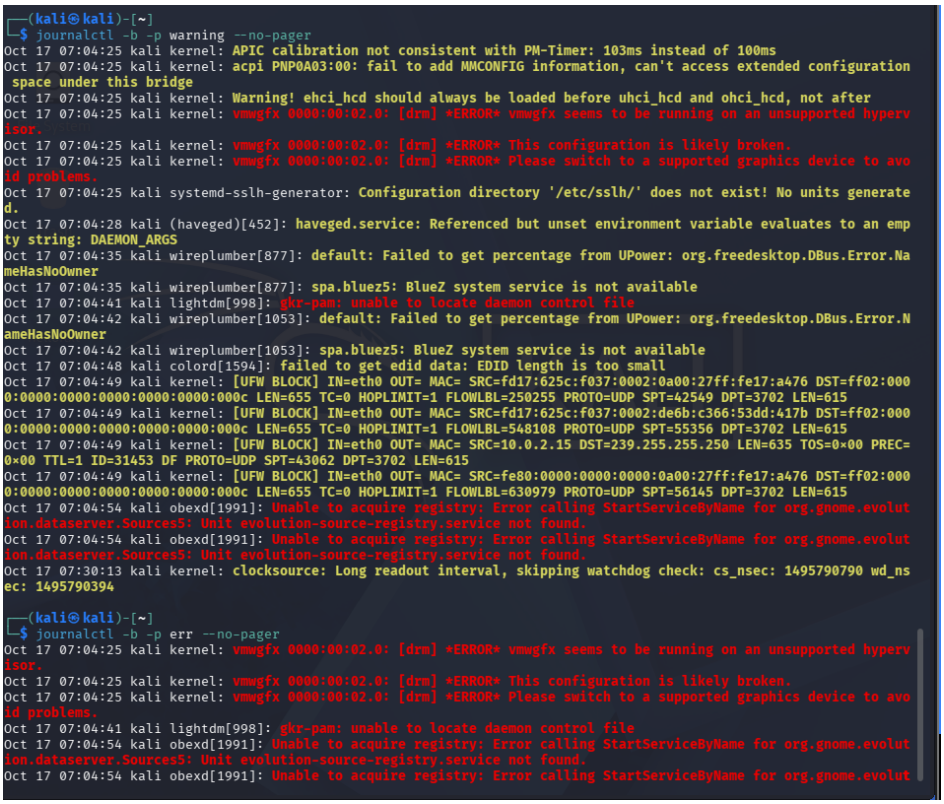


Figure 11: Warning and error priority journal entries.

6. Analysis of Error Messages

#	Message	Meaning	Assessment
1	`vmwgfx ... seems to be running on an unsupported hypervisor` (three lines)	The VMware graphics driver loaded while the VM runs on a different virtualization stack	Cosmetic unless display problems appear; document before changing graphics settings
2	`lightdm: gkr-pam: unable to locate daemon control file`	GNOME Keyring's PAM integration could not find its daemon control file at login	Relevant only if credential storage fails
3	`obexd: Unable to acquire registry ... evolution-source-registry.service not found` (twice)	A desktop component requested the Evolution data-source service, which is not installed	Harmless unless Evolution features are needed
4	`/var/log/syslog` absent	System uses the systemd journal	Expected behaviour, not a fault

Conclusion: an error message is an observation, not a confirmed root cause. The desktop was working normally, so none of these entries justified editing configuration or installing extra software.

7. Troubleshooting Case: Evidence File Would Not Save

Symptom

```
$ journalctl -b -n 100 --no-pager > evidence/recent-journal.txt 2>&1
zsh: no such file or directory: evidence/recent-journal.txt
```

A mistyped filename (`recentr-jor.txt`) produced the same error, which showed that the filename was not the cause.



Figure 12: Error saving the evidence file.

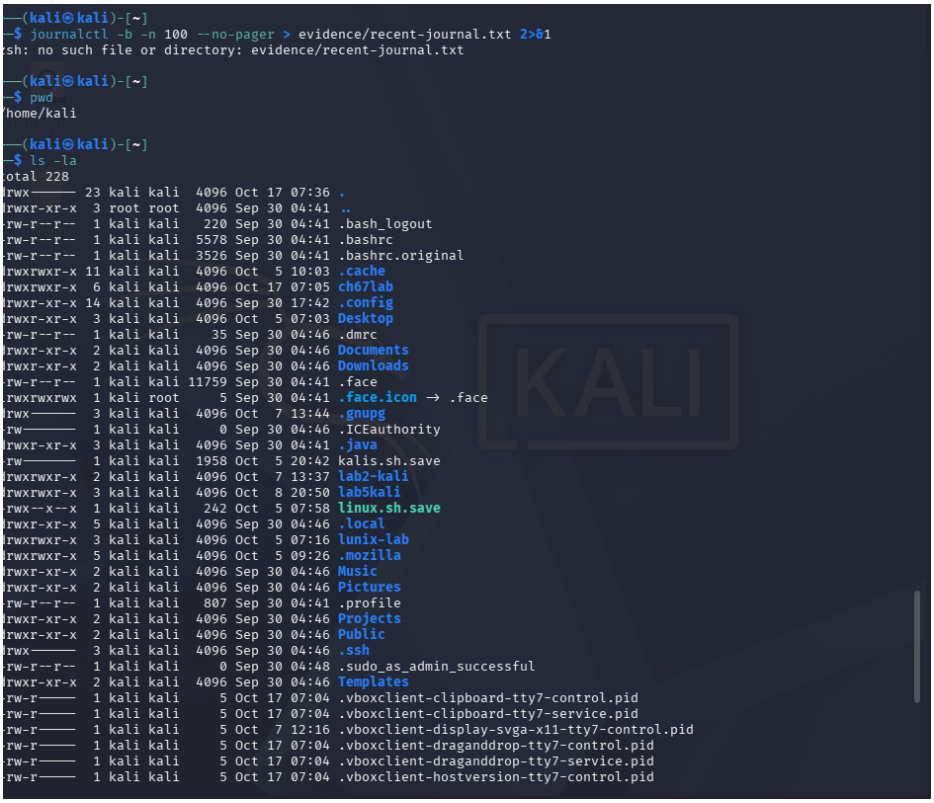


Figure 13: Working directory check showing ch67lab in home.

Investigation

1. `pwd` returned `/home/kali`, so the shell was in the home directory, not the lab workspace.
2. `ls -la` in `/home/kali` listed `ch67lab` but no `evidence` directory.
3. Relative paths are resolved from the current directory, and shell redirection cannot create missing parent directories.

Fix

```
cd ch67lab
ls -id evidence                                                    # directory confirmed (inode
    457714)
journalctl -b -n 100 --no-pager > evidence/recent-journal.txt 2>&1
ls -l evidence/recent-journal.txt
```

Verification

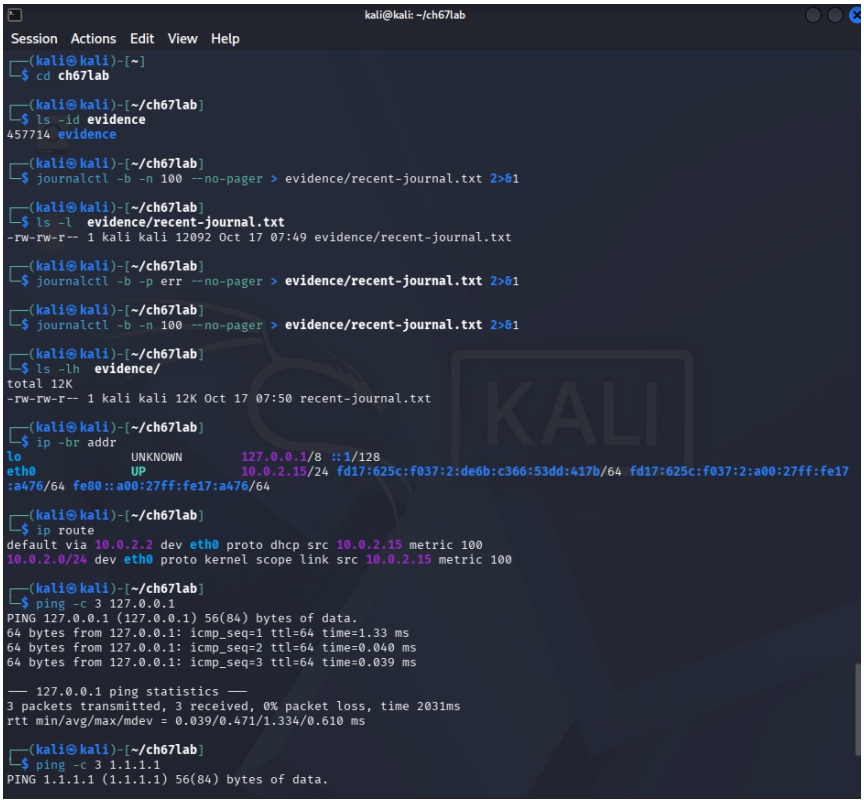


Figure 14: Evidence file saved and network retest.

- First save: `recent-journal.txt`, 12,092 bytes, 07:49.
- The error-only journal was then redirected to the same filename by mistake, which replaced the content. The 100-line journal was regenerated and `ls -lh evidence/` confirmed **recent-journal.txt, 12K, 07:50**.

> **Lessons:** verify your location before using relative paths, check the file after every redirect, and use a different filename for each output (for example `system-errors.txt` for error-priority logs) so one result does not overwrite another.

Navigation check

Running `cd ch67lab` from inside `ch67lab` failed with `no such file or directory`, and `ls -lh reports/network-ins.txt` failed from within `reports/` because the path was relative to the wrong directory. Both errors came from the same cause as the evidence problem, and `pwd`, `ls` and the shell prompt resolved them.

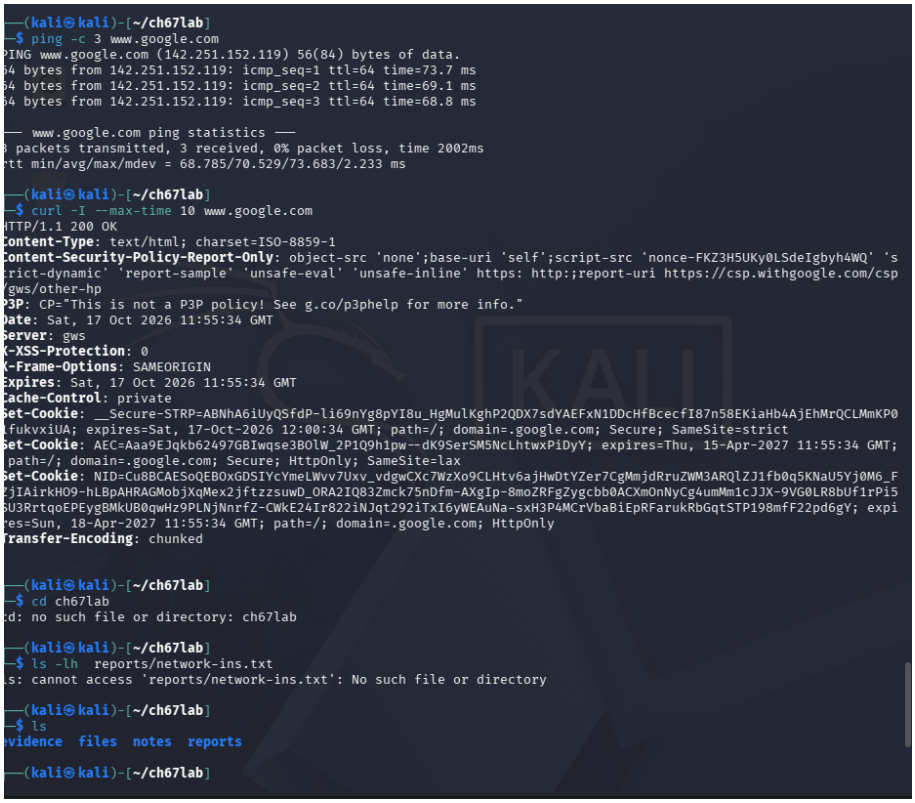


Figure 15: Ping and HTTP test to www.google.com.

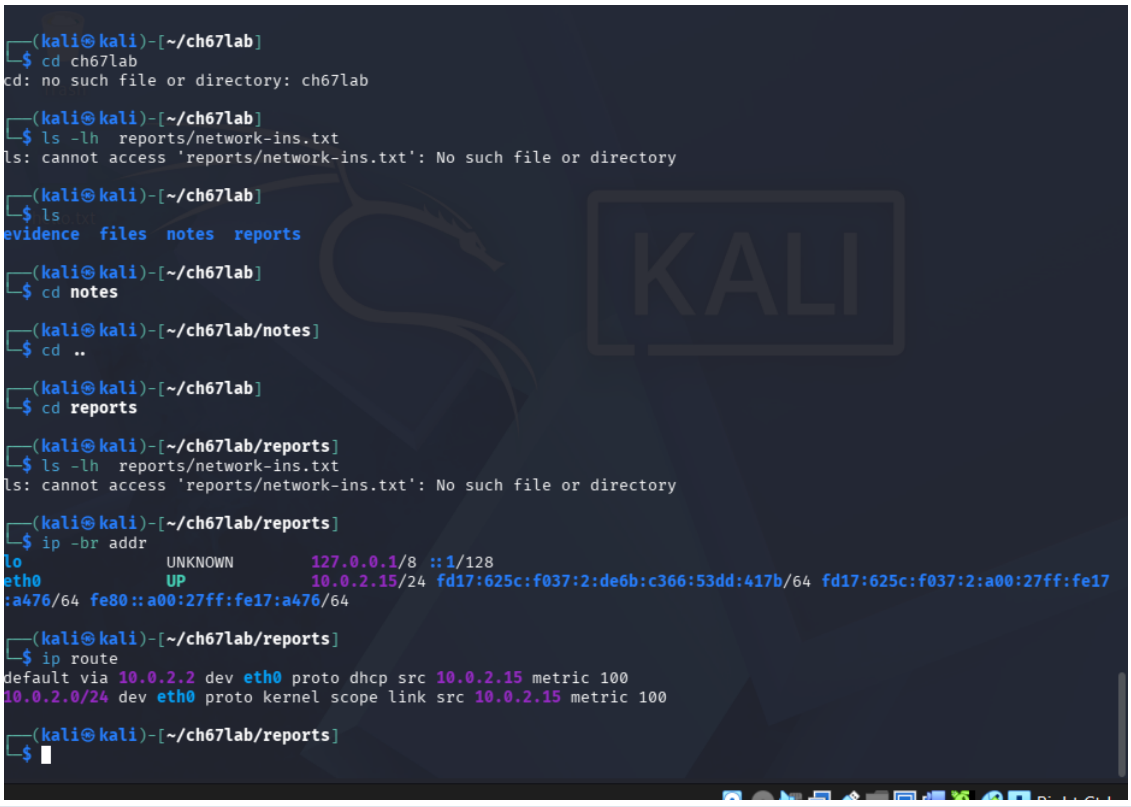


Figure 16: Directory navigation and path check.

8. Evidence Produced

File	Contents
`reports/baseline.txt`	Identity, date, hostname, OS release, kernel, uptime, memory, disk and network baseline

File	Contents
`evidence/recent-journal.txt`	Last 100 journal entries of the current boot (12 KB)

9. Summary of Findings

1. The system is a healthy Kali Linux Rolling 2026.2 VM with a 6.19.14 kernel running on KVM.
2. Disk usage on `/` is 86%, which is worth monitoring.
3. Networking works end to end: valid DHCP address, default route, DNS, ICMP and HTTP all succeeded.
4. NetworkManager manages the network; an apparent "inactive" result came from a misspelled service name.
5. The system uses the systemd journal instead of `/var/log/syslog`.
6. Graphics-driver, GNOME Keyring and Evolution registry messages were identified and documented without unverified changes.
7. The evidence-save failure was a working-directory problem, solved by diagnosing the path and verifying the result.

10. Skills Demonstrated

- System identification and resource inspection
- Using man pages, package metadata and installed documentation
- Layered network diagnosis: interface, address, route, DNS, ICMP, HTTP
- Filtering and reading the systemd journal by priority
- Shell redirection, relative paths and working-directory awareness
- Organising evidence and separating confirmed findings from possible causes
- Reading error messages to find typos and path mistakes

Conclusion

This lab showed that effective troubleshooting is about understanding output and confirming each assumption before acting. Every command either produced evidence or taught a lesson about paths, spelling or interpretation, and the final record is reproducible and backed by saved files.